

Web Application Penetration Test Report

Comprehensive Security Assessment & Risk Analysis

Target Application: OWASP Juice Shop (v20.2.0)

Assessment Date: May 2, 2026

Overall Risk Posture: HIGH

Prepared By: Akash Anil Horambe

Penetration Tester

akashhorambe19@gmail.com

<http://www.linkedin.com/in/akash-horambe>

Penetration Testing Report

1. Introduction

A Web Application Penetration Test (WAPT) was performed on the OWASP Juice Shop (v20.2.0) to assess the application's resilience to modern web attacks. Using a hybrid manual and automated approach based on the OWASP Top 10 methodology, critical security issues such as SQL Injection and IDOR were detected that allow for account takeover and data exfiltration.

2. Objective

The objective of the assessment was to assess the state of security and uncover vulnerabilities in **OWASP JUICE SHOP Web Application** and provide with a final security assessment report comprising vulnerabilities, remediation strategy and recommendation guidelines to help mitigate the identified vulnerabilities and risks during the activity.

3. Scope

This section defines the scope and boundaries of the project.

Application Name	OWASP JUICE SHOP
URL	http://localhost:3000/

3.1. Assessment Attribute(s)

Parameter	Value
Target Application	OWASP Juice Shop (v20.2.0)
Assessment Type	Web Application Penetration Testing (WAPT)
Starting Vector	External (Simulating attacks from the public internet)
Target Criticality	High (E-commerce platform processing PII and simulated financial data)
Assessment Nature	Cautious & Calculated
Proof of Concept(s)	Step-by-step reproduction steps and HTTP request/response screenshots attached for all findings.

3.2. Risk Calculation and Classification

Following is the risk classification:

Info	Low	Medium	High	Critical
No direct threat to host/ individual user account. Sensitive information can be revealed to the adversary.	Vulnerabilities may not have public exploit (code) available or cannot be exploited in the wild. Vulnerability observed may not have high rate of occurrence. Patch workaround released by vendor.	Vulnerabilities may not have public exploit (code) available or cannot be exploited in the wild. Patch/ workaround not yet released by vendor.	Vulnerabilities which can be exploited publicly, workaround or fix/ patch available by vendor.	Vulnerabilities which can be exploited publicly, workaround or fix/ patch may not be available by vendor.

Table 1: Risk Rating

Summary

Outlined is a Black Box Application Security assessment for **OWASP JUICE SHOP**

http://localhost:3000/
http://localhost:3000/*

Following section illustrates **Detailed** Technical information about identified vulnerabilities.

Total: 9 Vulnerabilities

Critical	High	Medium	Low
1	4	3	1

1. SQL Injection by injecting queries in login page

Reference No:	Risk Rating:
WEB_VUL_01	Critical CVSS v3.1: 9.8
Tools Used:	
Web Browser, Burp Suite Community Edition	
Vulnerability Description:	
It was observed that the authentication endpoint does not properly sanitize or parameterized user provided input before submitting to backend database as query. During authentication, the application transmits credentials to /rest/user/login by Json POST method. By injecting SQL payload (' OR 1=1 --) into email parameter of request the authentication query becomes TRUE at backend and authenticate user as administrator	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis & HTTP Request Manipulation via Burp Suite Proxy	
Vulnerable URLs / IP Address	
Front-end URL: http://localhost:3000/#/login	
Affected API Endpoint: POST http://localhost:3000/rest/user/login (Parameter: email)	
Implications / Consequences of not Fixing the Issue	
An unauthenticated attacker can bypass access control and gain admin access without valid credentials. This grants full administrative control over the application, attacker can view, modify and exfiltrate sensitive customer data (such as names, addresses, emails, and transaction histories).	
Suggested Countermeasures	
• Parameterized Queries: Avoid dynamic SQL string concatenation. Ensure the backend treats input strictly as data rather than executable SQL Command. • Input Validation: Enforce strict server-side validation on the email field using regular expressions • Principle of Least Privilege: Configure database service accounts with only the minimum read/write permissions required for routine operations • Web Application Firewall (WAF): Deploy a WAF with rulesets to detect and block common SQL injection signatures at the perimeter.	
References	
https://owasp.org/Top10/A03_2021-Injection/https://cwe.mitre.org/data/definitions/89.html	
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html	

Proof of concept:

1. **Intercept the Request:** Open Burp Suite, enable the intercepting proxy, and navigate to <http://localhost:3000/#/login>.

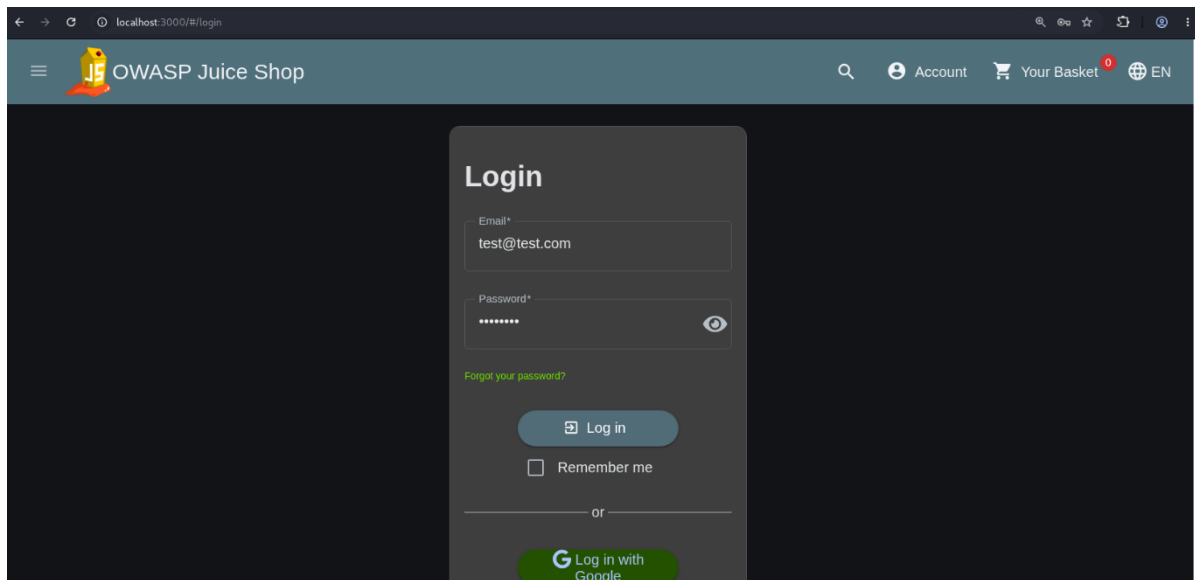


Figure 1 Login Page

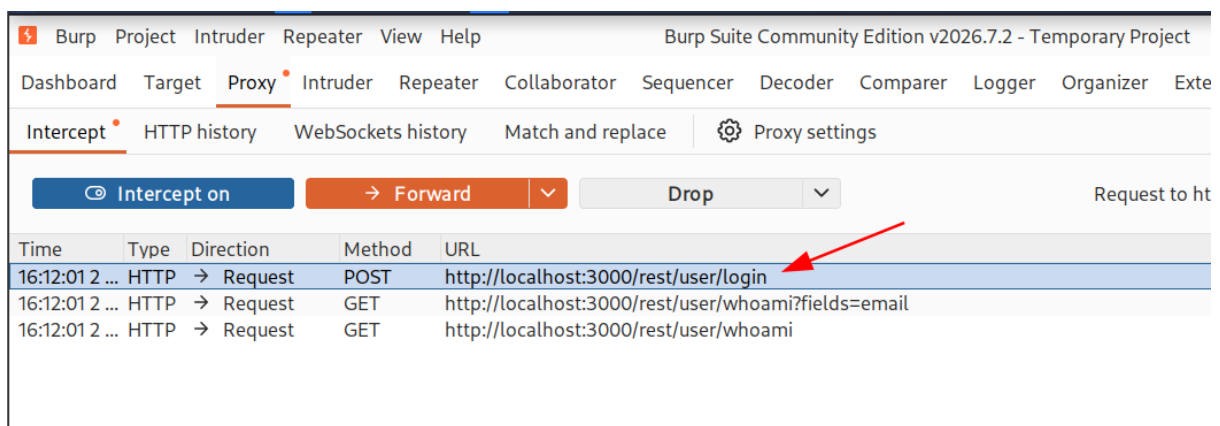


Figure 2 Burp Proxy Intercept Request

- Submit Credentials:** Enter an arbitrary email address (test@test.com) and password (password123), then submit the form.

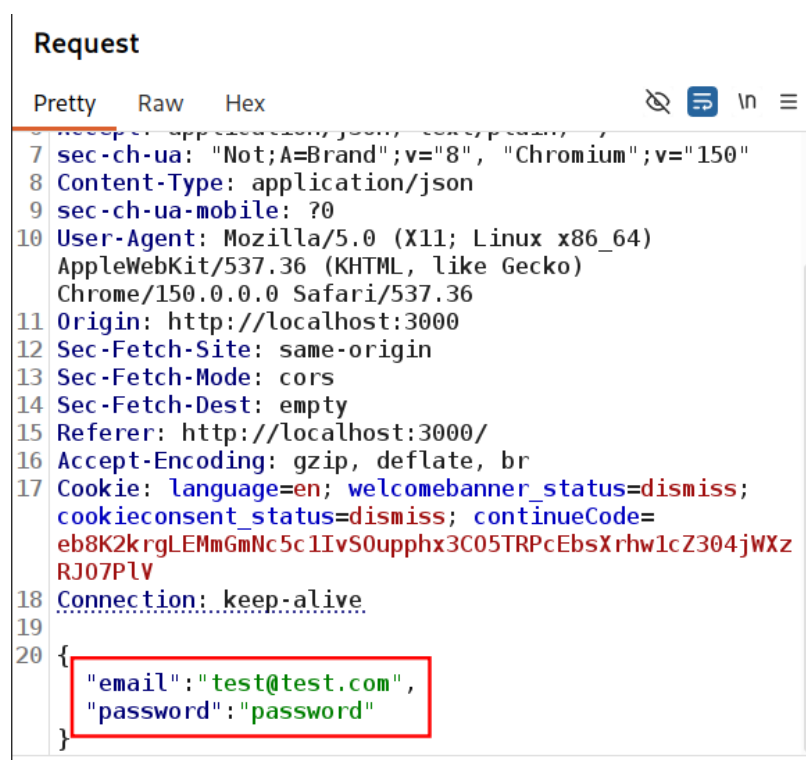
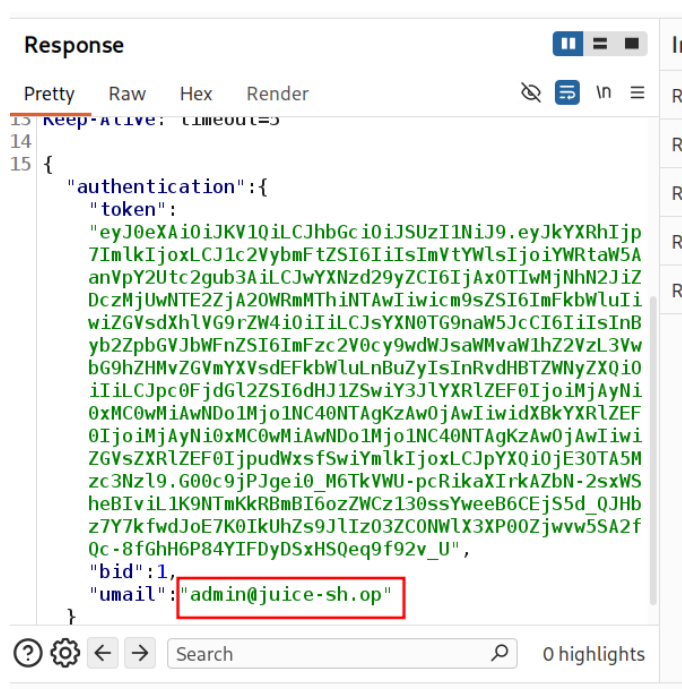


Figure 3 Request Interception

- 3. Inject the Payload:** Inject Payload and Forward the captured POST /rest/user/login request to Burp Repeater.

Figure 4 SQL Payload Injection



2. Unrestricted File Upload.

Reference No:	Risk Rating:
WEB_VUL_02	High CVSS v3.1: 7.5
Tools Used:	
Web Browser, Burp Suite Community Edition	
Vulnerability Description:	
It was observed that the application's file upload functionality at the customer complaint endpoint relies on client side restriction and weak file-type validation. The front end restrict the file upload to specific file formats (such as .pdf) but the backend handling the upload fails to validate the file extension. As a result, an attacker can bypass client-side checks and upload any file extensions or payloads directly to the file system.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis & Intercepted Request Manipulation via Burp Suite Proxy	
Vulnerable URLs / IP Address	
Front-end URL: http://localhost:3000/#/complain	
Affected API Endpoint: POST http://localhost:3000/file-upload (Parameter: file)	
Implications / Consequences of not Fixing the Issue	
Allowing unrestricted file uploads exposes the server and its clients to multiple risks. If uploaded files are stored in a web-accessible directory with execution permissions, an attacker could upload a server-side script to achieve Remote Code Execution (RCE).also file-system pollution/denial of service through oversized files.	
Suggested Countermeasures	
Server-Side Extension Whitelisting: Enforce a strict allow-list of safe extensions (e.g., only .pdf, .png, .jpg) on the server. Never rely solely on client-side JavaScript or HTTP Content-Type headers provided by the client.	
MIME Type & Magic Byte Validation: Inspect the file header bytes (magic numbers) on the backend to verify that the file structure matches its declared format.	
Offload and Isolate Uploads: Store uploaded assets in a dedicated, external storage container (e.g., an S3 bucket or isolated file server) outside the application web root.	
Disable Execution Permissions: Ensure the destination directory on the server is mounted with no-execution flags (noexec) and web server execution handlers (e.g., Node/PHP/CGI execution) are disabled for that path.	
Enforce Maximum File Size: Set hard upload size limits both at the reverse proxy (e.g., NGINX) and application layer to mitigate denial-of-service risks.	
References	
https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload	
https://cwe.mitre.org/data/definitions/434.html https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html	

Proof of concept:

1. **Locate the Upload Form:** Navigate to the "Customer Feedback" or "File a Complaint" section at <http://localhost:3000/#/complain>.

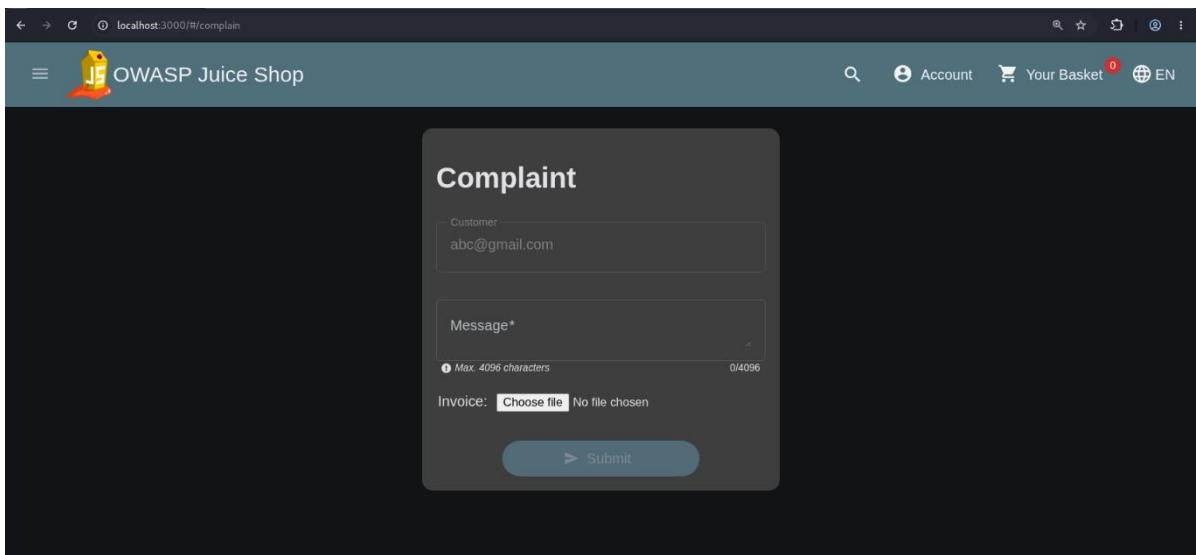


Figure 6 Complaint Page

2. **Observe Client-Side Enforcement:** Attempt to select a non-PDF file (such as a benign .txt or .html file) using the standard file picker. Note that the browser dialog or front-end validation restricts file selection to .pdf formats.

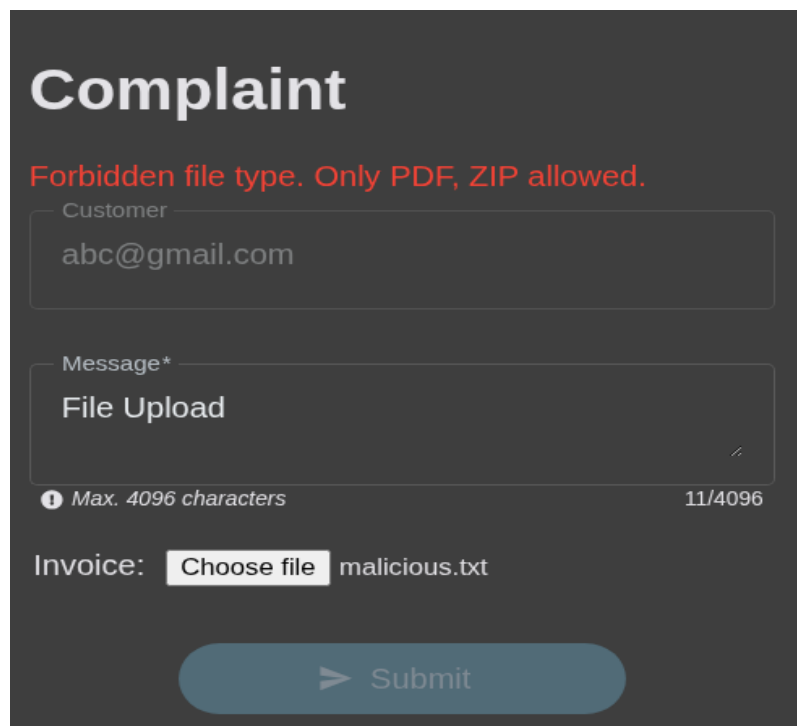


Figure 7 Error on uploading wrong file extension

3. **Intercept the Submission:** Select an allowed dummy file (e.g., test.pdf), enable intercept in Burp Suite, enter the required message fields, and click **Submit**.

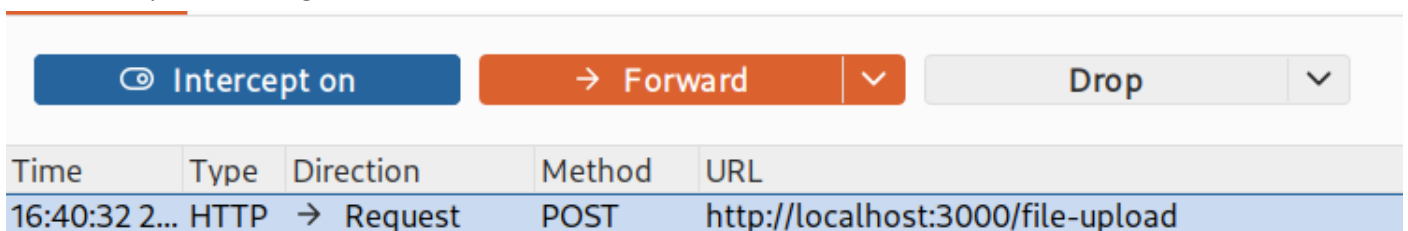


Figure 8 Request Intercept

Request

Pretty Raw Hex



```
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImIkJoyNSwidXNlcm5hbWUiOiIiLCJlbWFPbCI6ImFiY0BnbWFpbC5jb20iLCJwYXNzd29yZCI6ImQ4NTc4ZWrmODQ10GNlMDZmYmM1YmI3NmE1OGM1Y2E0Iiwicm9sZSI6ImNlc3RvbWVYIiwizGVsdXhlVG9rZW4iOiIiLCJ5YXN0TG9naW5JcCI6IjAuMC4wLjAiLCJwcm9maWxlSW1hZ2UiOiIvYXNzZXRzL3B1YmXPYy9pbWFnZXMvdXBsb2Fkcy9kZWZhdWx0LnN2ZyIsInRvdHBTZWNYZXQiOiIiLCJpc0FjdG12ZSI6dHJ1ZSwiY3JlYXRLZEF0IjoimjAyNi0xMC0wMiAxMTowNzowOC4zMTCgKzAw0jAwIiwidXBkYXRLZEF0IjoimjAyNi0xMC0wMiAxMTowNzowOC4zMTCgKzAw0jAwIiwizGVsZXRLZEF0IjpudWxsfsWiYmlkIjo2LCJpYXQiOjE3OTA5MzkyMzV9.b2hDA0WwKUha5087CvDse31cllJX4z32EUgfjsekYCVTY5LIxmDrCg2jaZ9tUfUa4rtc2GP4tWDNkITX_p_kPFci701YwFrhhVIhlG8pWDT3j5A-UtLrVTPSYRPvdqRiW3h4zfY1GqpScCiCC8ZqJQasWYQh6bj1aUkg23lp8E
19 Connection: keep-alive
20
21 -----WebKitFormBoundaryY2ipwSRilbIM7SfS
22 Content-Disposition: form-data; name="file"; filename="file.pdf"
23 Content-Type: application/pdf
24
25 File
26
27
28 -----WebKitFormBoundaryY2ipwSRilbIM7SfS--
29
```

Figure 9 Analyzing Request

4. **Modify the Multipart Form Data:** Forward the captured POST /file-upload request to Burp Repeater. Locate the Content-Disposition header and adjust the filename parameter.

Request

Pretty Raw Hex



```
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImIkJoyNSwidXNlcm5hbWUiOiIiLCJlbWFPbCI6ImFiY0BnbWFpbC5jb20iLCJwYXNzd29yZCI6ImQ4NTc4ZWrmODQ10GNlMDZmYmM1YmI3NmE1OGM1Y2E0Iiwicm9sZSI6ImNlc3RvbWVYIiwizGVsdXhlVG9rZW4iOiIiLCJ5YXN0TG9naW5JcCI6IjAuMC4wLjAiLCJwcm9maWxlSW1hZ2UiOiIvYXNzZXRzL3B1YmXPYy9pbWFnZXMvdXBsb2Fkcy9kZWZhdWx0LnN2ZyIsInRvdHBTZWNYZXQiOiIiLCJpc0FjdG12ZSI6dHJ1ZSwiY3JlYXRLZEF0IjoimjAyNi0xMC0wMiAxMTowNzowOC4zMTCgKzAw0jAwIiwidXBkYXRLZEF0IjoimjAyNi0xMC0wMiAxMTowNzowOC4zMTCgKzAw0jAwIiwizGVsZXRLZEF0IjpudWxsfsWiYmlkIjo2LCJpYXQiOjE3OTA5MzkyMzV9.b2hDA0WwKUha5087CvDse31cllJX4z32EUgfjsekYCVTY5LIxmDrCg2jaZ9tUfUa4rtc2GP4tWDNkITX_p_kPFci701YwFrhhVIhlG8pWDT3j5A-UtLrVTPSYRPvdqRiW3h4zfY1GqpScCiCC8ZqJQasWYQh6bj1aUkg23lp8E
19 Connection: keep-alive
20
21 -----WebKitFormBoundaryY2ipwSRilbIM7SfS
22 Content-Disposition: form-data; name="file"; filename="malicious.txt"
23 Content-Type: application/pdf
24
25 File
26
27
28 -----WebKitFormBoundaryY2ipwSRilbIM7SfS--
29
```

Figure 10 Modify Request

5. **Analyze the Server Response:** Forward the request. The server accepts the file and responds with an HTTP/1.1 204 No Content or HTTP/1.1 200 OK, confirming that the server accepted and saved the non-allowlisted file format without rejection.

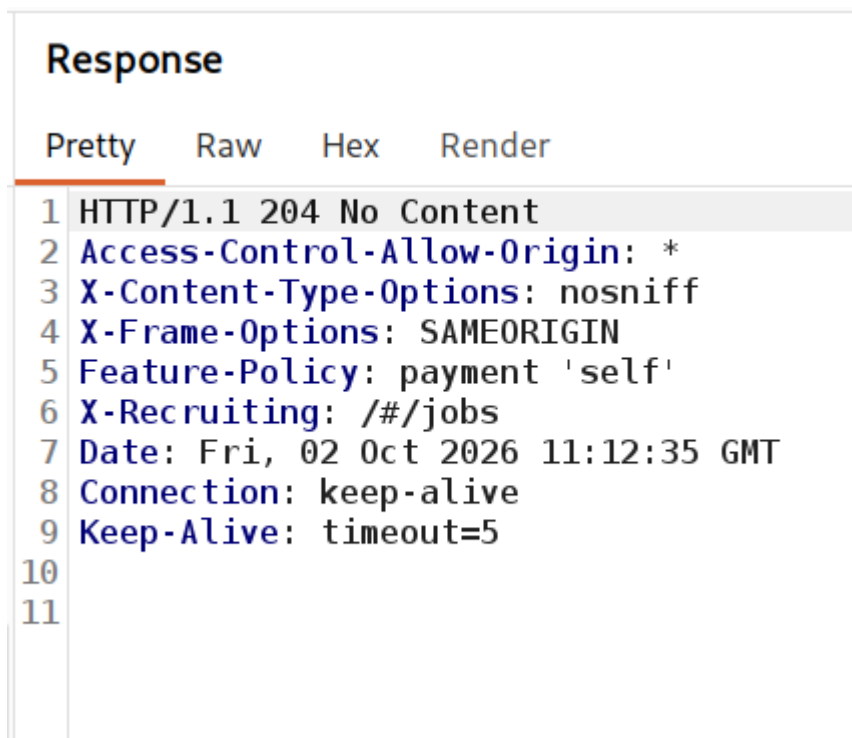


Figure 11 Response from server

3. IDOR (View/Edit Another Basket).

Reference No:	Risk Rating:
WEB_VUL_03	High CVSS v3.1: 7.5
Tools Used:	
Web Browser, Burp Suite Community Edition	
Vulnerability Description:	
It was observed that the application relies on user controlled object reference to retrieve or update cart contents without server side authorization. While navigating to basket (#/basket), the application makes API request to server GET /rest/basket/{bid}, where bid is integer represents the users basket id . The backend fails to verify if the request is made by the authorized user. By simply changing this bid attacker can read and change basket items of other users.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis & HTTP Parameter Tampering via Burp Suite Proxy	
Vulnerable URLs / IP Address	
Front-end URL: http://localhost:3000/#/basket	
Affected API Endpoint: GET http://localhost:3000/rest/basket/{bid}	
Implications / Consequences of not Fixing the Issue	
An attacker can systematically enumerate integer IDs to view the products, quantities, prices, and personal cart contents belonging to any customer on the platform (Broken Object-Level Authorization /IDOR). Furthermore, tampering with update requests (e.g., BasketId in BasketItems) allows adversaries to inject, modify, or delete items in another customer's basket, compromising transactional integrity and user privacy.	
Suggested Countermeasures	
• Object-Level Authorization Checks: Enforce strict server-side validation verifying that the authenticated user identity (extracted securely from the session cookie or JWT token) has explicit ownership over the requested basket entity before returning data. • Do Not Trust Client-Supplied Identifiers: Avoid accepting the bid / BasketId directly as an arbitrary route parameter or request body variable. Instead, query the user's active basket strictly by resolving the user ID stored inside the validated server-side session or JWT claims. • Use Non-Sequential Identifiers (UUIDs): Replace easily predictable auto-incrementing integer IDs (1, 2, 3...) with cryptographically random UUIDv4 identifiers to prevent trivial parameter enumeration.	
References	
https://owasp.org/Top10/A01_2021-Broken_Access_Control/https://cwe.mitre.org/data/definitions/639.html https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.htmlhttps://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/	

Proof of concept:

1. **Populate the Cart & Intercept Traffic:** Log into an account, add an item to the shopping cart, and navigate to the basket page at http://localhost:3000/#/basket with Burp Suite Intercept enabled.

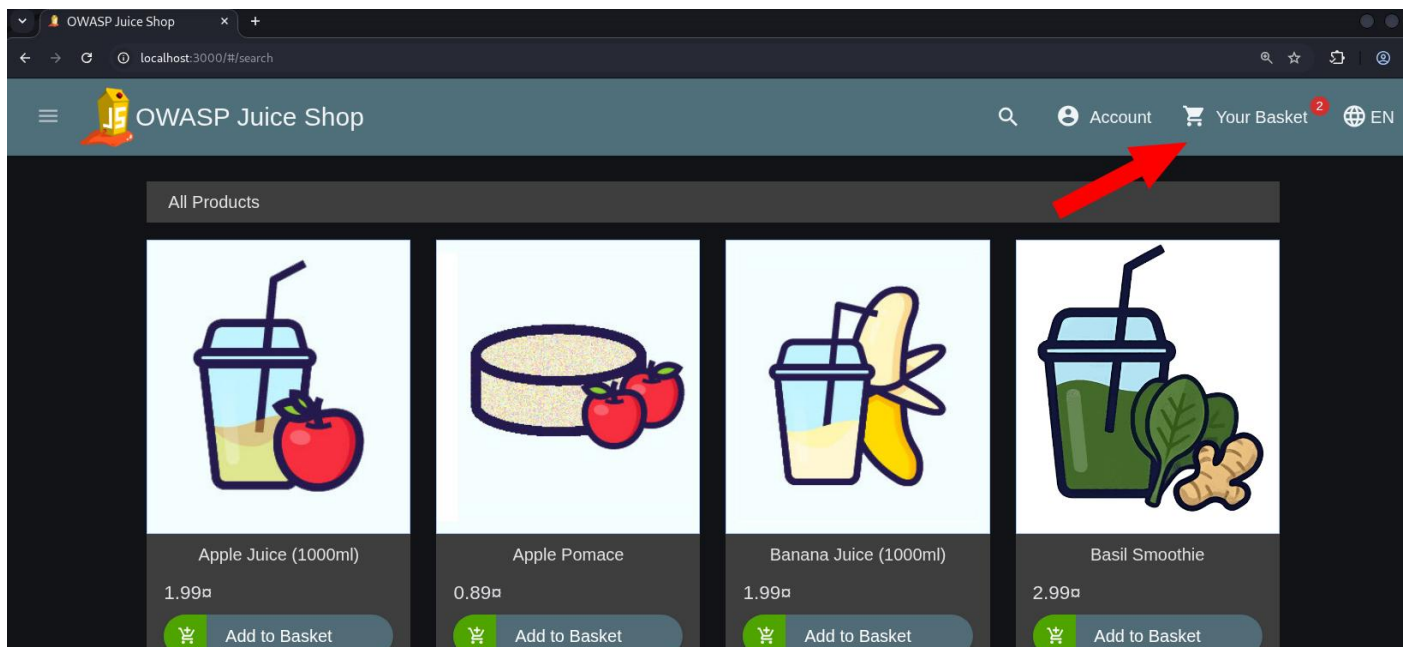


Figure 12 Basket Page

2. **Identify the Direct Reference:** In the Burp HTTP History or active intercept tab, locate the GET request fetching cart items:

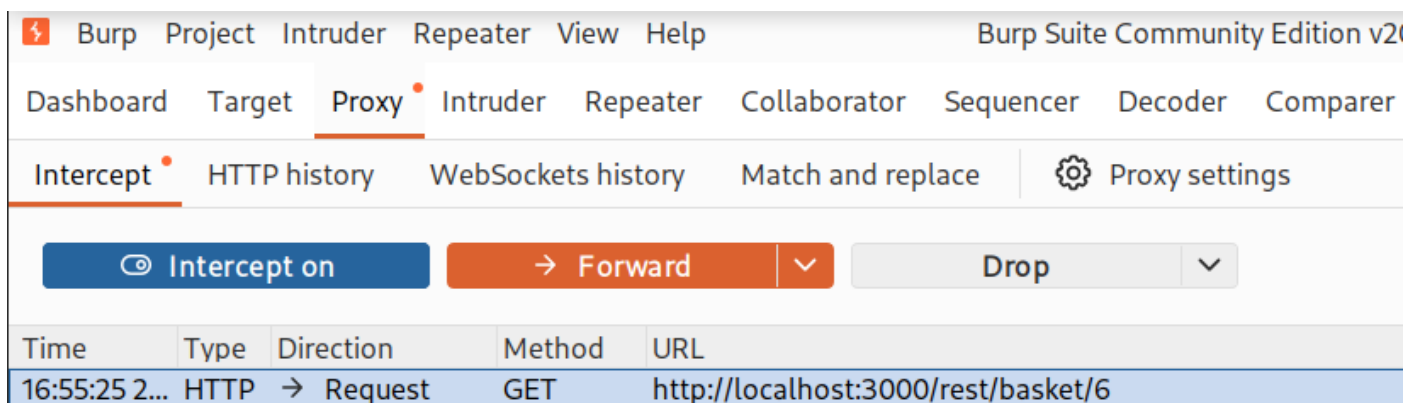


Figure 13 Intercept Request

3. **Tamper with the Basket ID:** Send the request to Burp Repeater. Alter the changing URI parameter from 6 to 5 (or another sequential integer representing a target basket)

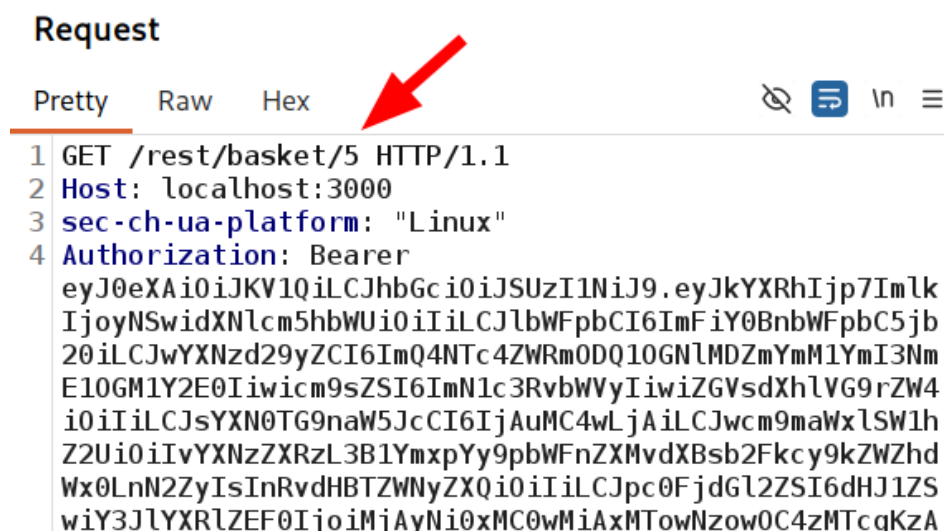
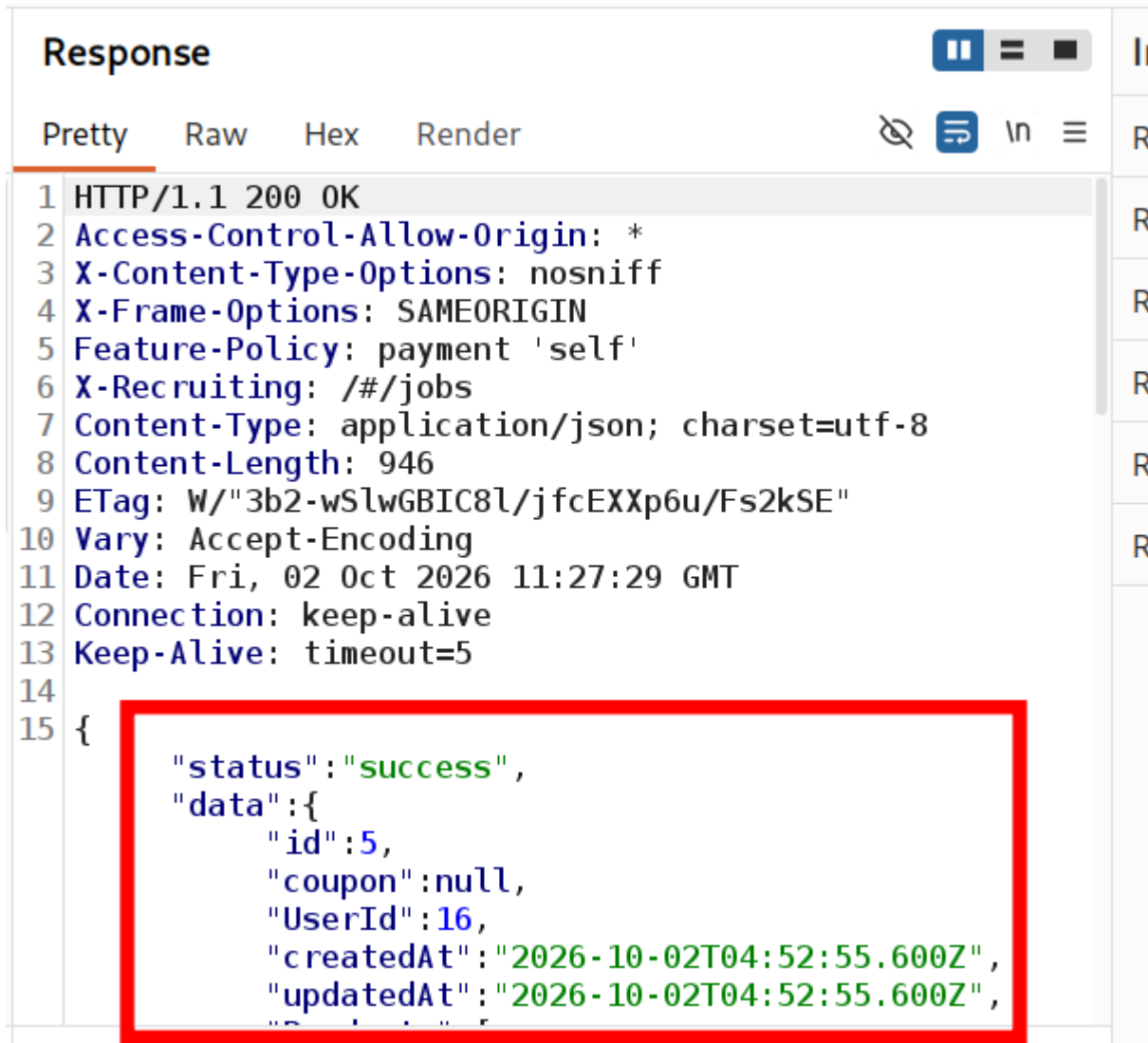


Figure 14 Tamper Request

4. **Inspect the Response Data:** Click **Send**. The server responds with HTTP/1.1 200 OK containing the JSON payload of another customer's basket, including product IDs, item names, and quantities



```
Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 946
9 ETag: W/"3b2-wSlwGBIC8l/jfcEXXp6u/Fs2kSE"
10 Vary: Accept-Encoding
11 Date: Fri, 02 Oct 2026 11:27:29 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "status": "success",
  "data": {
    "id": 5,
    "coupon": null,
    "UserId": 16,
    "createdAt": "2026-10-02T04:52:55.600Z",
    "updatedAt": "2026-10-02T04:52:55.600Z",
    "product": {
      "id": 1,
      "name": "Product 1",
      "price": 100,
      "quantity": 1
    },
    "product": {
      "id": 2,
      "name": "Product 2",
      "price": 200,
      "quantity": 1
    },
    "product": {
      "id": 3,
      "name": "Product 3",
      "price": 300,
      "quantity": 1
    }
  }
}
```

Figure 15 Response from server (IDOR Successful)

4. Broken Access Control (Administration Page).

Reference No:	Risk Rating:
WEB_VUL_04	High CVSS v3.1: 7.5
Tools Used:	
Web Browser, Browser Developer Tools	
Vulnerability Description:	
It was observed that the application hides the Administration dashboard for non-admin users. But the application fails to enforce Role-Based Access Control on the actual endpoint. Because the backend do not validate users' privilege level upon direct request hence unauthorized standard user can access administrative panel.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis, Client-Side Source Code Review (JavaScript Route Mapping), and Forced Browsing	
Vulnerable URLs / IP Address	
Front-end URL: http://localhost:3000/#/administration	
Implications / Consequences of not Fixing the Issue	
Any user can access the administration portal. This exposes sensitive internal business data, including the complete registered user accounts, customer email addresses, and unredacted customer feedback. If associated administrative write-APIs (like deleting users) also lack authorization checks, this could escalate to complete application defacement and data destruction.	
Suggested Countermeasures	
• Server-Side Role Validation: Enforce strict access controls on the backend API. Every request to an administrative endpoint (e.g., /api/Users/) must explicitly verify that the requesting user's JWT or session token contains an admin role claim. • Fail Securely: If a user without the correct privileges requests an administrative endpoint, the server must return an HTTP 403 Forbidden or HTTP 401 Unauthorized response, preventing any data from loading. • Do Not Rely on Obscurity: Assume all client-side JavaScript (including hidden routes and endpoints) can and will be read by attackers.	
References	
https://owasp.org/Top10/A01_2021-Broken_Access_Control/https://cwe.mitre.org/data/definitions/284.html https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html	

Proof of concept:

1. **Client-Side Reconnaissance:** Open the application in a standard web browser without logging in. Open the Browser Developer Tools (F12) and navigate to the **Debugger/Sources** tab.

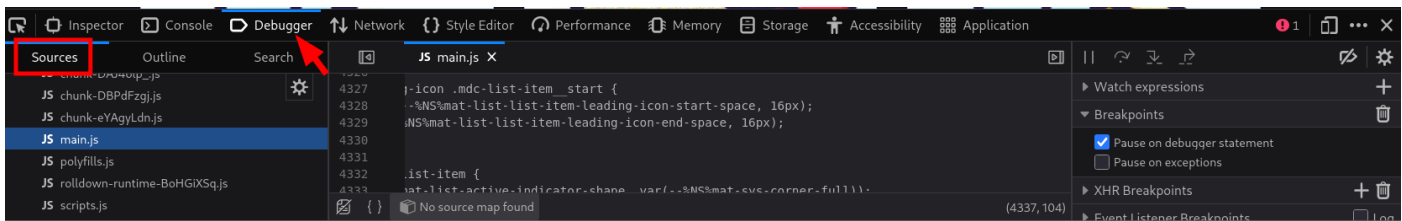


Figure 16 Debugger Tool

2. **Analyze Route Maps:** Search through the client-side JavaScript bundles (e.g., main.js or similar compiled Angular files for routing keywords like path:.

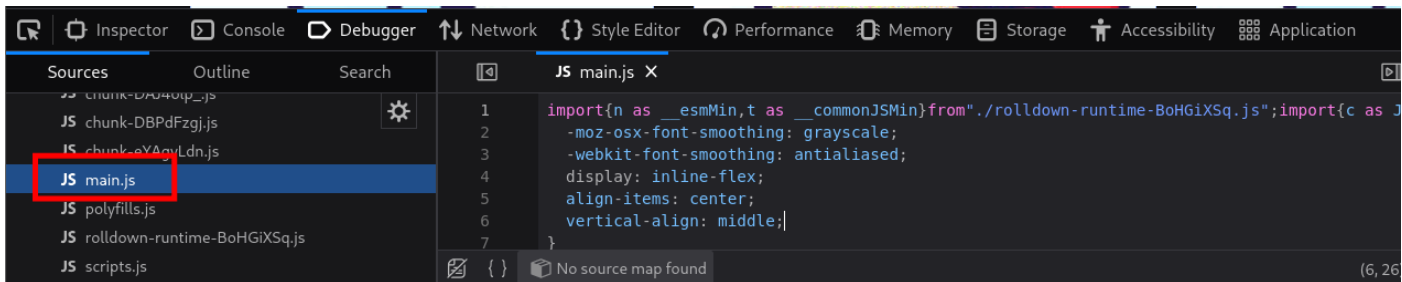


Figure 17 Main.js File

3. **Discover the Hidden Route:** Observe that a route definition exists for path: 'administration', which maps to an administration component, despite no visual link existing in the main navigation bar.



Figure 18 Path for admin panel

4. **Execute Forced Browsing:** In the browser's address bar, append the discovered route to the base URL, navigating directly to <http://localhost:3000/#/administration>. The application successfully loads the Administration dashboard

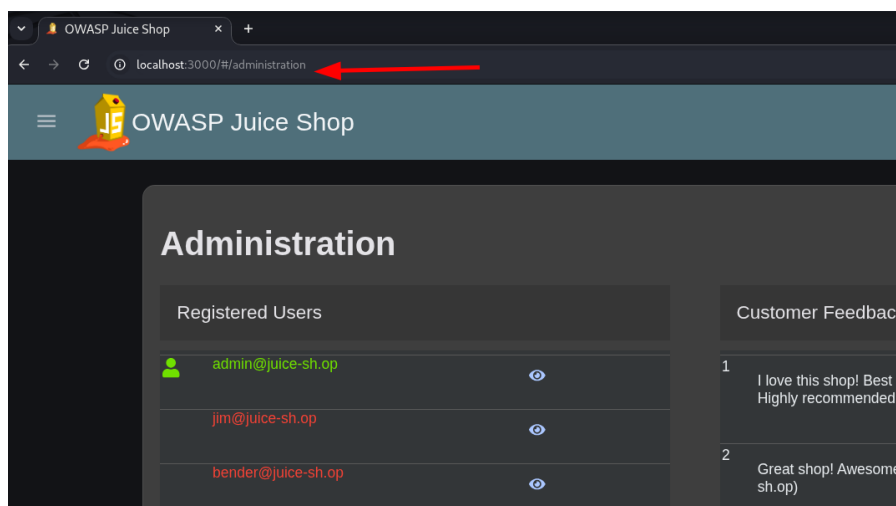


Figure 19 Admin Panel

5. Reflected / DOM XSS (Search Bar)

Reference No:	Risk Rating:
WEB_VUL_05	High CVSS v3.1: 7.1
Tools Used:	
Web Browser	
Vulnerability Description:	
It was observed that the applications search functionality is vulnerable to Cross Site Scripting (XSS). When users submit a query the application dynamically reflect user input in DOM to display "Search Results for: {input}" message. Because the front end do not sanitize the input before rendering into browser window, attacker can inject java script payloads	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis, Client-Side Execution Testing	
Vulnerable URLs / IP Address	
http://localhost:3000/#/search?q=	
Implications / Consequences of not Fixing the Issue	
An attacker can craft a malicious URL containing the JavaScript payload and distribute it to victims via phishing emails or social engineering. If a logged-in user clicks the link, the script executes in their browser context. This allows the attacker to steal sensitive authentication data (like JWTs stored in Local Storage or non-HttpOnly cookies), silently log keystrokes, modify the page appearance (defacement), or force the victim's browser to perform unauthorized actions (like changing account passwords) on their behalf.	
Suggested Countermeasures	
• Context-Aware Output Encoding: Ensure that all user-supplied data reflected in the UI is strictly treated as text, not executable code. • Content Security Policy (CSP): Implement a strict CSP HTTP response header to restrict the sources from which scripts can be loaded and executed. Specifically, deny the execution of inline scripts (unsafe-inline) and eval(). • Input Validation: Implement an allow-list approach on the backend API to strip or reject input containing <, >, / or script tags before it is ever processed or returned to the client.	
References	
https://owasp.org/Top10/A03_2021-Injection/https://cwe.mitre.org/data/definitions/79.html https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html	

Proof of concept:

1. **Locate the Search Input:** Open the application in a web browser (**http://localhost:3000**) and locate the magnifying glass / search bar at the top of the interface.

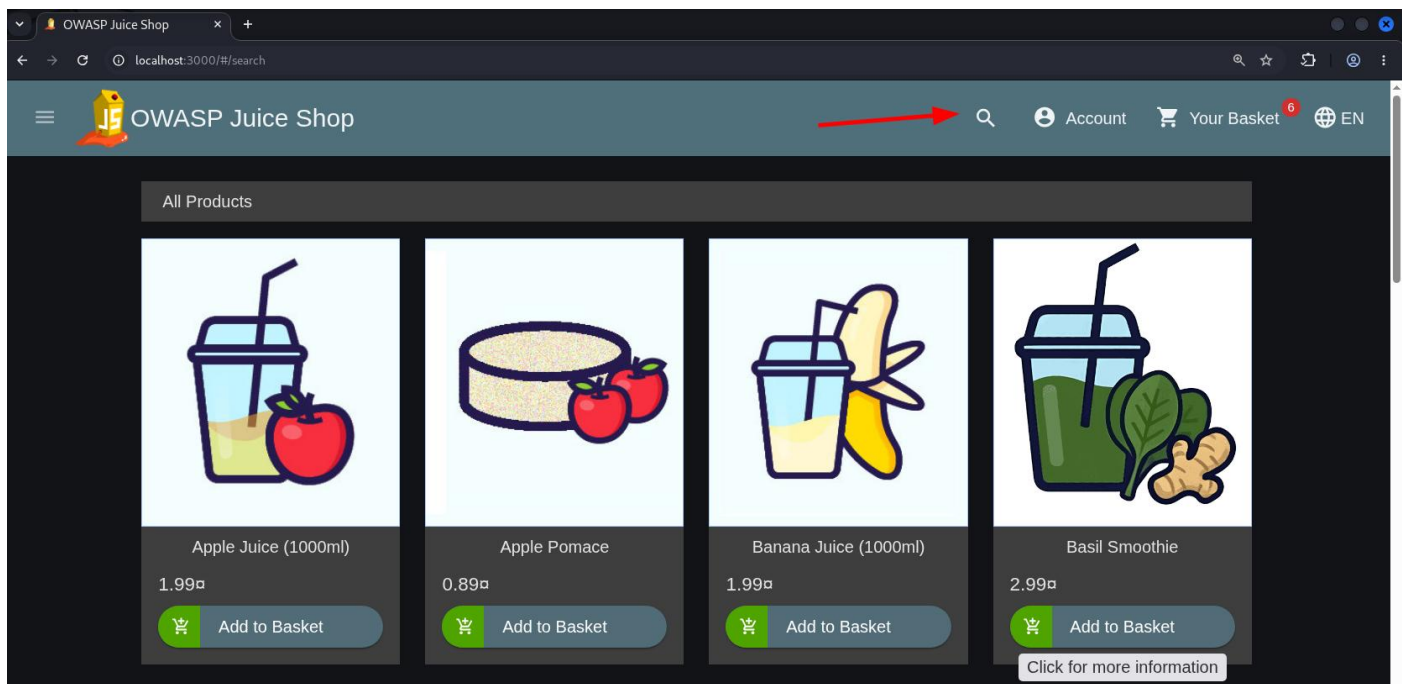


Figure 20 Search Bar

2. **Craft the Payload:** Instead of a standard search term like "Apple Juice", enter a standard XSS vector designed to test for HTML interpretation. Since Juice Shop uses Angular (which blocks basic `<script>` tags by default), use an `iframe` payload to bypass basic filters:
`<iframe src="javascript:alert('XSS')">`

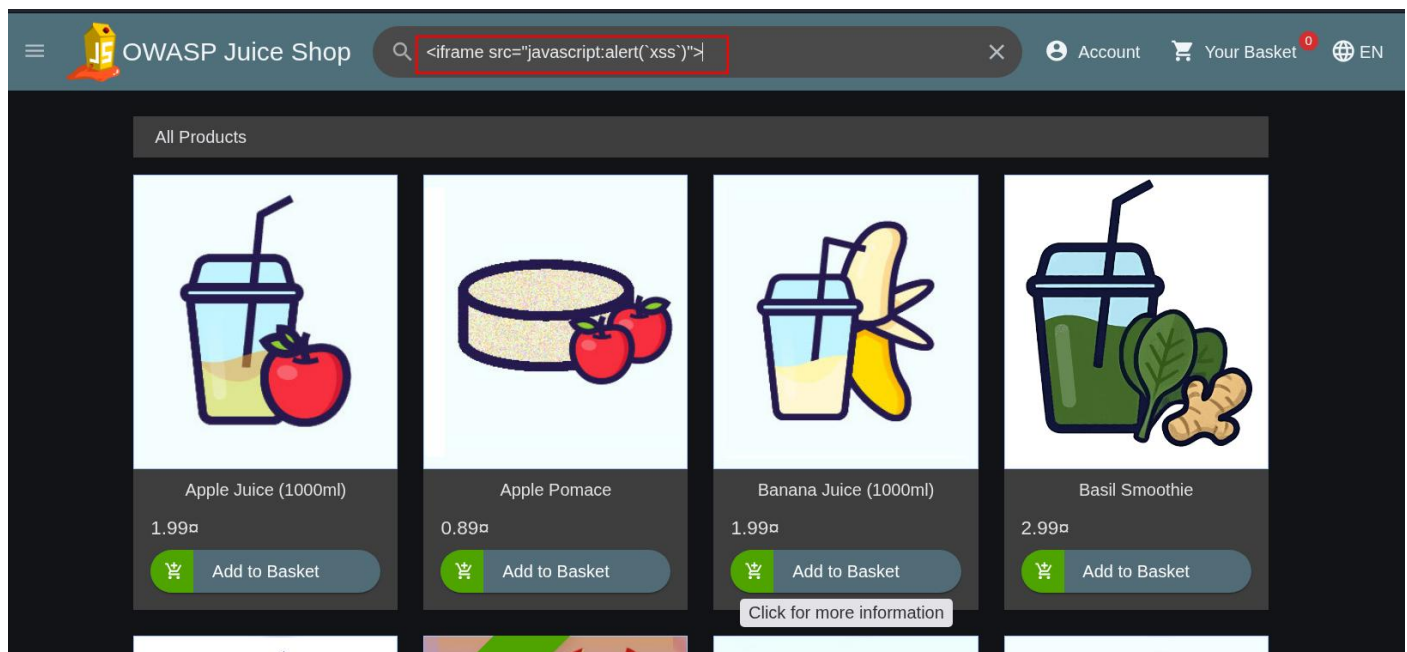


Figure 21 XSS Payload

3. **Observe the Client-Side Execution:** The application route changes to `/#/search?q=<iframe src="javascript:alert('XSS_Execution_Confirmed')">`. Instead of safely printing the raw text on the screen, the browser interprets the `iframe` tag and executes the embedded JavaScript, triggering a pop-up alert box containing the text `XSS`. This proves that arbitrary script execution is possible in the client's browser session.

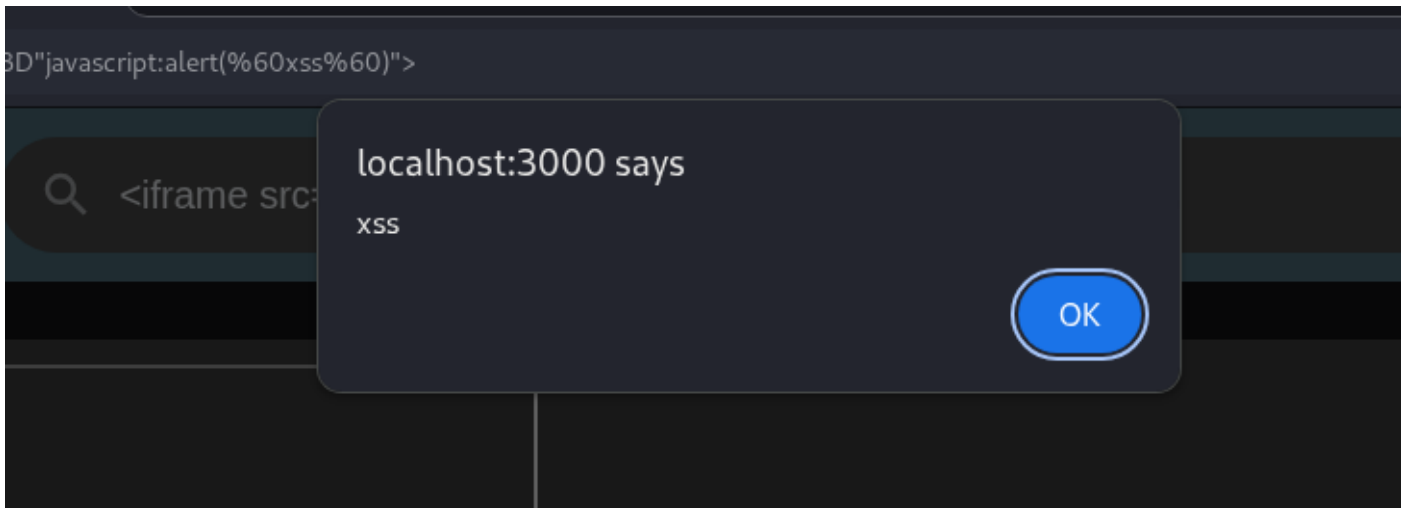


Figure 22 XSS Successful

6. Business Logic Flaw (Negative Basket Pricing)

Reference No:	Risk Rating:
WEB_VUL_06	Medium CVSS v3.1: 6.5
Tools Used:	
Web Browser, Burp Suite Community Edition	
Vulnerability Description:	
It was observed that the applications shopping basket API has several business logic flaws. When adding product to the cart or updating item quantities, the backend API (/api/basket) trust user provided quantity parameter without enforcing value boundary. By intercepting the http request and modifying the quantity to negative number (e.g.-10) the application calculates negative subtotal for that item. This lowers the total cost of the cart, allowing user to purchase items for free.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis & HTTP Parameter Tampering via Burp Suite Proxy	
Vulnerable URLs / IP Address	
Front-end URL: http://localhost:3000/#/basket Affected API Endpoint: POST http://localhost:3000/api/BasketItems/ and PUT http://localhost:3000/api/BasketItems/{id}	
Implications / Consequences of not Fixing the Issue	
If exploited in a real-world e-commerce environment, this logic flaw directly results in financial loss and inventory theft. Attackers can manipulate their order totals to exactly \$0.00 to bypass credit card processing entirely, or create negative totals to potentially exploit refund/credit mechanics, acquiring physical or digital goods without payment.	
Suggested Countermeasures	
• Strict Server-Side Input Validation: Never trust client-side constraints (like HTML5 min="1" attributes). The backend API must explicitly validate that the quantity parameter is an integer strictly greater than zero (> 0) before processing the database transaction. • Business Logic Checks at Checkout: Before finalizing any order, the backend must recalculate the total cart value and verify that the final amount is greater than zero and matches the expected sum of all valid product prices. • Schema Enforcement: Update the database schema for the BasketItems table to include a CHECK (quantity > 0) constraint, ensuring that the database itself rejects negative inventory updates.	
References	
https://cwe.mitre.org/data/definitions/840.html https://cwe.mitre.org/data/definitions/1284.html	

Proof of concept:

1. **Populate the Cart:** Log into a standard user account. Browse the application and add a high-priced item (e.g., "OWASP Juice Shop Hoodie" for \$49.99) to your basket.

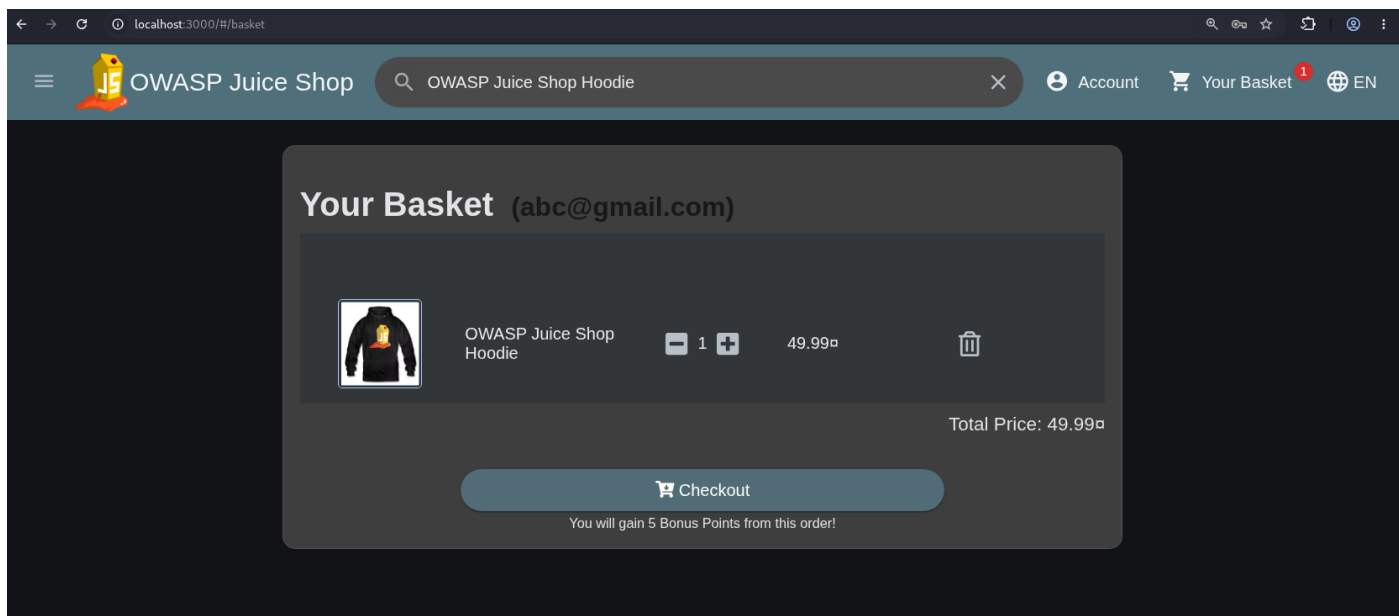


Figure 23 Adding high price item in basket

2. **Intercept the Attack Vector:** Find a cheaper item (e.g., "Apple Juice" for \$1.99). Click "Add to Basket", but intercept the resulting HTTP request in Burp Suite.

Intercept on
→ Forward
▼
Drop
▼

Time	Type	Direction	Method	URL
17:27:40 2...	HTTP	→ Request	POST	http://localhost:3000/api/BasketItems/

Request

Pretty
Raw
Hex

```

19 3suV97T8s89qx8jv4B6EkQJuv3HSdmBg31_lzq8ihdDtUats5K15lXtcoWYJGhm0uCucYbTWZvu0-
20  GKGbKyEnqgK7rK7oJjoYQkQ2-m3MteTVCiTvLLwWGbqpZk
21 Connection: keep-alive
   {
     "ProductId":1,
     "BasketId":"6",
     "quantity":1
   }

```

Figure 24 Intercept Request

3. **Tamper with the Quantity:** The captured POST `/api/BasketItems/` request will contain a JSON body with the product ID, basket ID, and quantity. Modify the quantity field to a negative number large enough to offset the price of the Hoodie (e.g., -30):

Figure 25 Modify Request

- 4. Forward and Verify:** Forward the modified request to the server. The server responds with HTTP/1.1 200 OK.

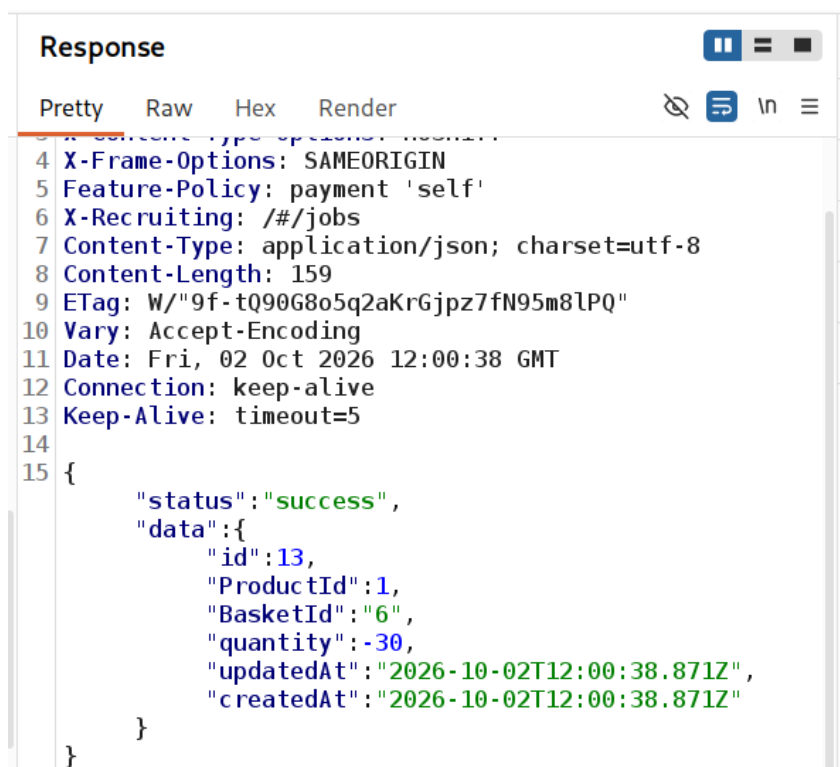


Figure 26 Server Response

5. **Observe the Logic Failure:** Navigate to <http://localhost:3000/#/basket> in your browser. The cart will display the Hoodie (\$39.99) and -25 units of Apple Juice (-\$49.75). The total checkout price is now recalculated to a negative amount (or exactly \$0.00 depending on application routing limits), allowing you to proceed through the checkout process and "purchase" the Hoodie without paying.

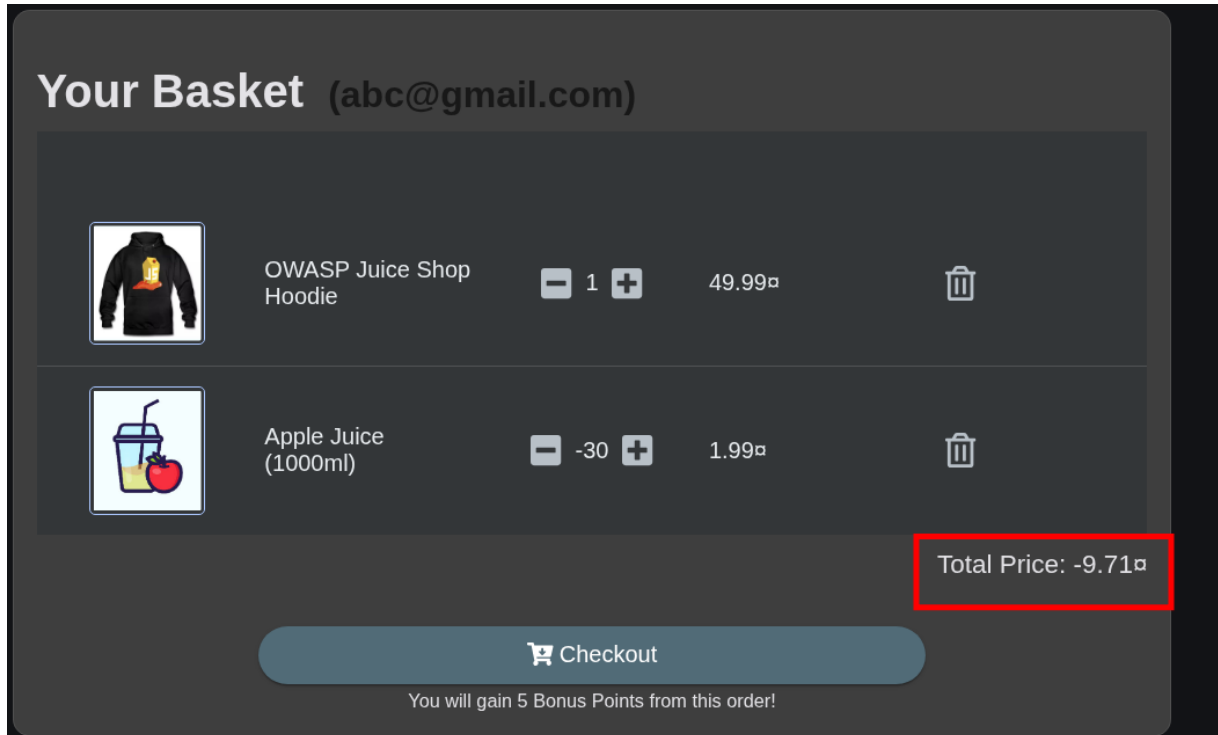


Figure 27 Total Price In Negative

7. Weak Authentication (Forgot Password / Security Questions)

Reference No:	Risk Rating:
WEB_VUL_07	Medium CVSS v3.1: 5.3
Tools Used:	
Web Browser, Open-Source Intelligence (OSINT) / Search Engines	
Vulnerability Description:	
It was observed that the application relies on knowledge-based authentication via static security question for password recovery. The answers to this questions (e.g., "What is your eldest sibling's middle name?", "What's your favorite pet's name?") are insecure. They can be predicted , guessed or discovered through OSINT on social media or wiki , In this environment, target accounts can be compromised simply by researching the user's public background.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis & OSINT Profiling	
Vulnerable URLs / IP Address	
http://localhost:3000/#/forgot-password	
Implications / Consequences of not Fixing the Issue	
An unauthenticated attacker can bypass the primary password and reset a target user's credentials without requiring access to their registered email address or a secondary physical device. This leads to immediate Account Takeover (ATO), granting the attacker unauthorized access to the victim's personal data, transaction history, and account privileges.	
Suggested Countermeasures	
• Deprecate Security Questions: Remove Knowledge-Based Authentication entirely. It is fundamentally flawed as "secret" knowledge is rarely truly secret in the modern internet era. • Out-of-Band Password Reset: Implement a secure reset flow that emails a cryptographically strong, single-use, time-limited token (e.g., expiring in 15-30 minutes) embedded in a reset link to the user's verified email address • Multi-Factor Authentication (MFA): Enforce MFA across all accounts. If a password reset is initiated, the user must still provide their second factor (e.g., TOTP app or SMS code) before the reset is finalized. • Rate Limiting & Alerting: Apply strict rate limiting to the password reset endpoint to prevent automated guessing attacks against the security question endpoint, and alert users via email whenever a reset is requested.	
References	
https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/ https://cwe.mitre.org/data/definitions/640.html	
https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html	

Proof of concept:

1. **Locate the Reset Mechanism:** Navigate to the login page and click the "Forgot Password?" link, directing you to <http://localhost:3000/#/forgot-password>.

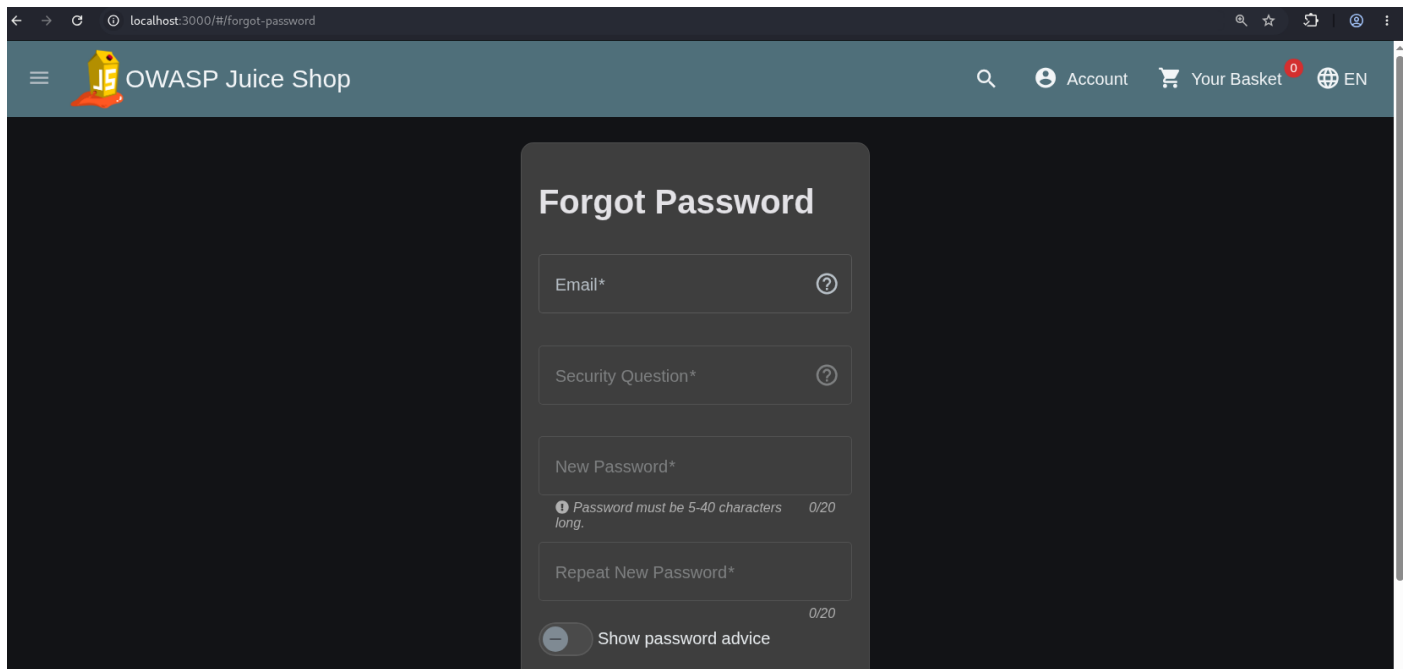


Figure 28 Forgot Password Page

2. **Target an Account:** Enter the email address of a known target account, such as emma@juice-sh.op, into the "Email" field to reveal their specific security question.

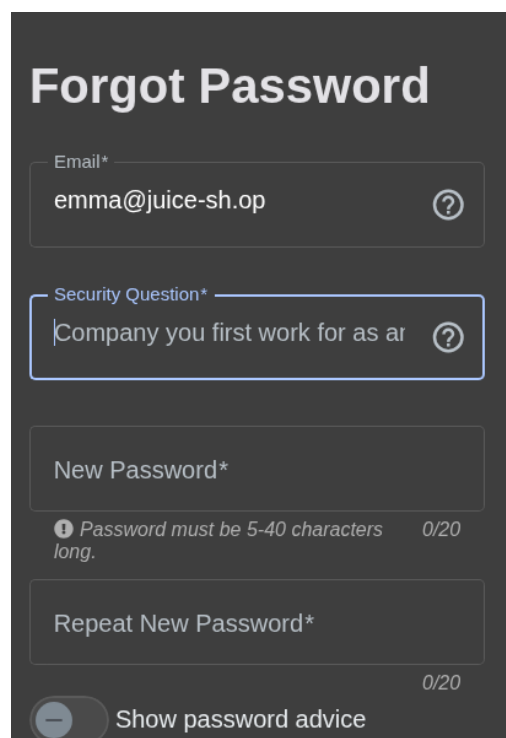
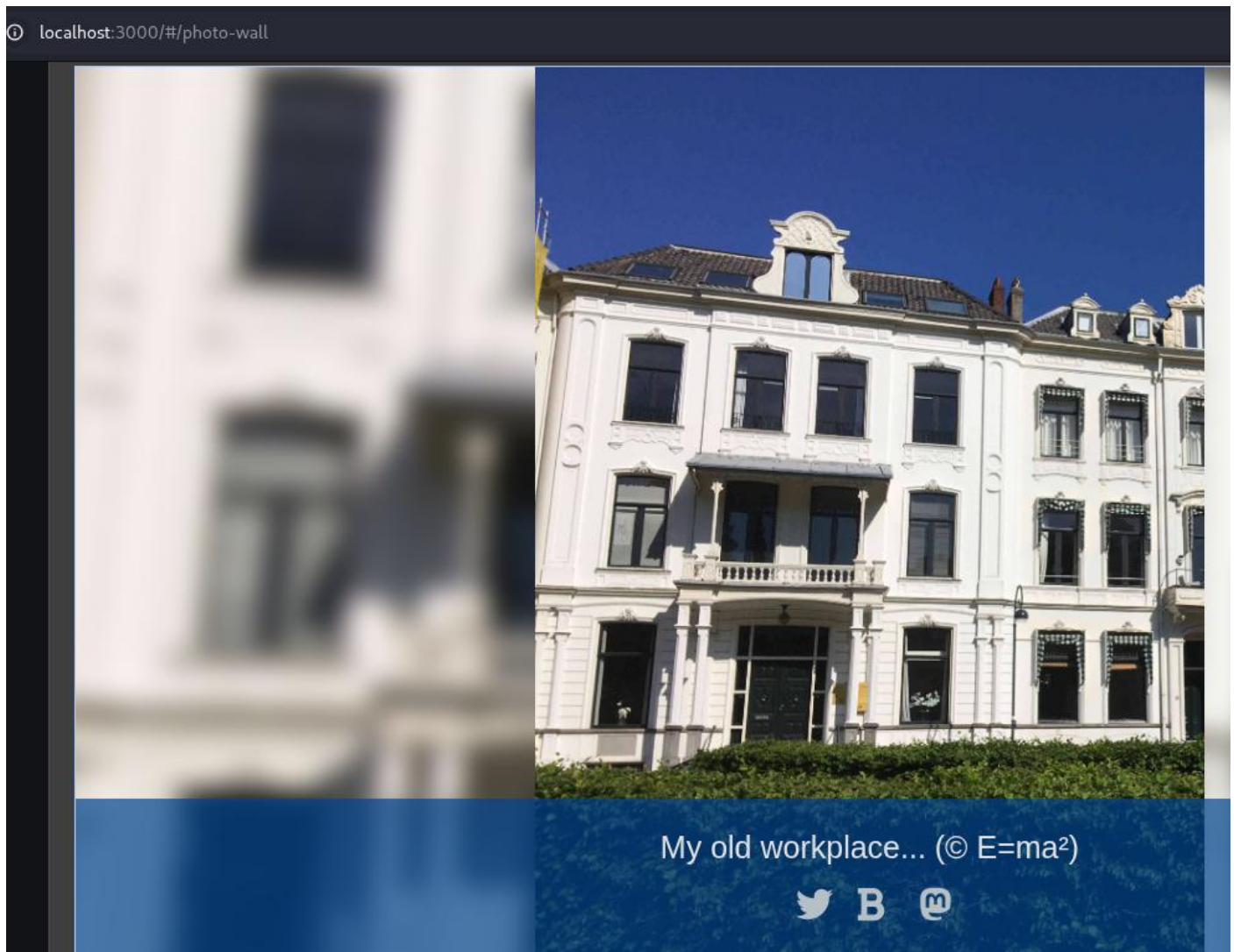


Figure 29 Target Emma's Account

3. **Conduct OSINT Reconnaissance:** The application reveals the security question for this user. Leave the application and use a search engine (like Google) to research the target's background. Because this specific target is based on a well-known pop culture character, a quick wiki search reveals the character's background lore and associated names I found emma's photo in photo wall section of application.



4. Bypass the Challenge: Return to the Juice Shop password reset page. Enter the discovered answer into the "Security Answer" field.

Forgot Password

Email*
emma@juice-sh.op ?

Security Question*
..... ?

New Password*
.....

! Password must be 5-40 characters long. 6/20

Repeat New Password*
..... 6/20

☐ Show password advice

Figure 30 Bypass Security question

5. **Set a New Password:** Provide a new password (e.g., Hacked123!), confirm it, and click "Change".

Forgot Password

Your password was successfully changed.

Email* ?

Security Question* ?

New Password*

! Password must be 5-40 characters long. 0/20

Repeat New Password*

0/20

Show password advice

Figure 31 Setting New Password

6. **Verify Account Takeover:** The server responds with HTTP/1.1 200 OK. Navigate back to the login page and successfully authenticate as `emma@juice-sh.op` using your newly created password, confirming full account compromise.

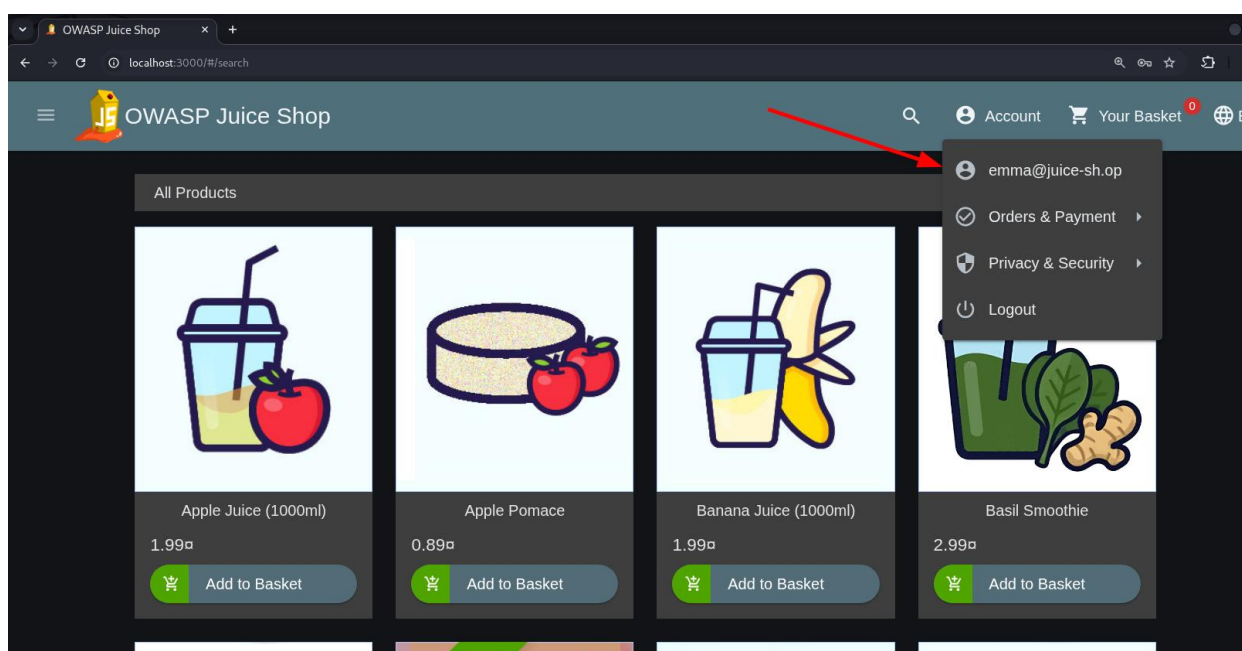


Figure 32 Logged into Emma's Account

8. Directory Listing / Exposed FTP Directory

Reference No:	Risk Rating:
WEB_VUL_08	Medium CVSS v3.1: 5.3
Tools Used:	
Web Browser, Directory Enumeration Tool Gobuster	
Vulnerability Description:	
It was observed that the applications web server is misconfigured which allows directory indexing. When user navigates to /ftp directory it displays html index of all files and folders present within the directory exposing internal assets (e.g. backup files, documents etc.) to the public internet.	
Vulnerability Identified by / How It Was Discovered	
Automated Directory Fuzzing & Manual Forced Browsing	
Vulnerable URLs / IP Address	
http://localhost:3000/ftp	
Implications / Consequences of not Fixing the Issue	
While directory listing does not directly execute code, it is a critical reconnaissance vector for attackers (Information Disclosure). Exposing this directory allows malicious actors to download sensitive corporate data, unearth forgotten backup files containing old credentials, or steal password vaults (e.g., .kdbx files). This leaked intelligence significantly lowers the barrier for orchestrating secondary, higher-severity attacks against the organization.	
Suggested Countermeasures	
• Disable Directory Indexing: Turn off the auto-indexing feature globally in the web server configuration. • Store Sensitive Files Outside the Web Root: Never store backups, internal documents, or password databases within the public web directory (/public, /var/www/html, etc.). Move them to an isolated, non-routable directory. • Implement Access Controls: If the /ftp directory is intended for legitimate file downloads, enforce strict authentication and authorization checks before allowing access to the route. • Return Generic Errors: Ensure the server is configured to return a generic HTTP 403 Forbidden or HTTP 404 Not Found when a user attempts to access a directory without a default document.	
References	
https://owasp.org/Top10/A05_2021-Security_Misconfiguration/https://cwe.mitre.org/data/definitions/548.html	
https://cwe.mitre.org/data/definitions/538.html	

Proof of concept:

1. **Reconnaissance:** Run a directory brute-force tool (like Gobuster) against the target, or manually inspect client-side JavaScript for file path references. Discover the existence of the /ftp path.

```
$ gobuster dir -u http://localhost:3000 -w /usr/share/wordlists/dirb/common.txt
--exclude-length 9393

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://localhost:3000
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] Exclude Length: 9393
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

apis (Status: 500) [Size: 2410]
api (Status: 500) [Size: 2408]
assets (Status: 301) [Size: 156] [→ /assets/]
ftp (Status: 200) [Size: 11307]
```

Figure 33 Gobuster Directory Finding

2. **Execute Forced Browsing:** Open a standard web browser and navigate directly to `http://localhost:3000/ftp`. Do not log in.

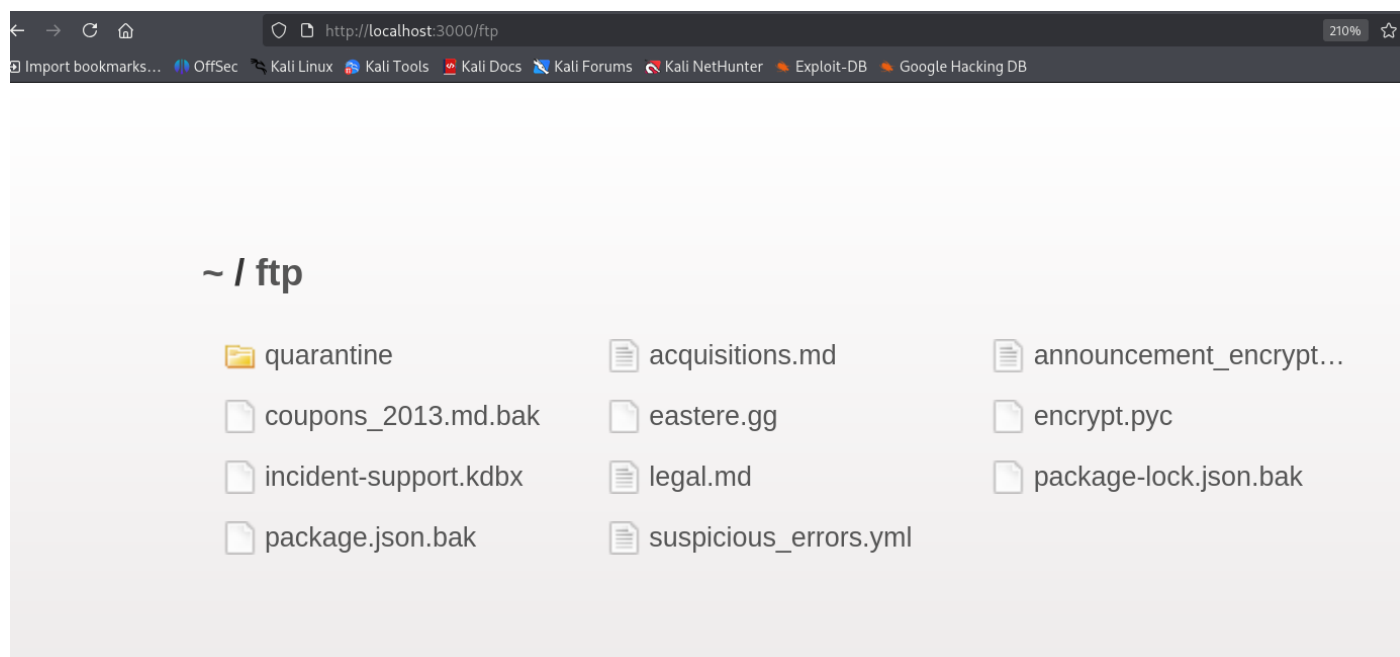


Figure 34 FTP directory

3. **Access Sensitive Data: Browse the exposed files. Observe highly sensitive internal documents, such as acquisitions.md (confidential corporate strategy), coupons_2013.md.bak (old backup files), and incident-support.kdbx (a KeePass password database).**

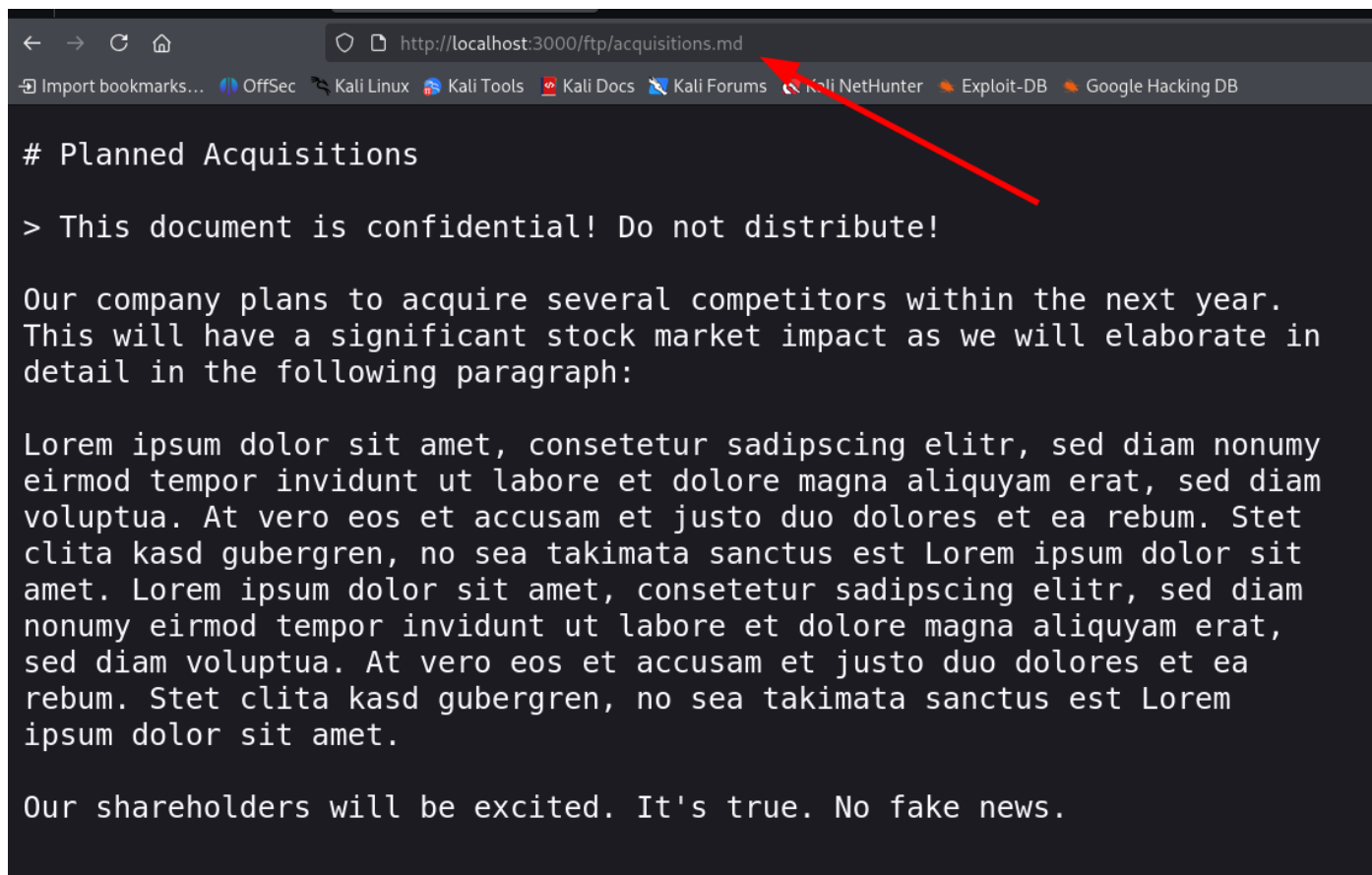


Figure 35 Sensitive Data Exposure

9. Missing Security Headers

Reference No:	Risk Rating:
WEB_VUL_09	Low / Informational CVSS v3.1: 3.7
Tools Used:	
Web Browser, Curl tool	
Vulnerability Description:	
It was observed that the application's HTTP response failed to implement standard HTTP security headers, Specifically, the server does not enforce Content-Security-Policy (CSP), X-Frame-Options, Strict-Transport-Security (HSTS), or X-Content-Type-Options. Without this headers, the application relies entirely on the client browsers default security mechanisms.	
Vulnerability Identified by / How It Was Discovered	
Manual HTTP Response Analysis & Automated Baseline Scanning	
Vulnerable URLs / IP Address	
http://localhost:3000	
Implications / Consequences of not Fixing the Issue	
While missing headers do not directly lead to a standalone compromise, their absence significantly weakens the application's security posture. For example, lacking X-Frame-Options allows an attacker to embed the site in a malicious iframe to orchestrate Clickjacking attacks. Lacking a strict Content-Security-Policy makes it vastly easier for an attacker to successfully execute Cross-Site Scripting (XSS) payloads by loading malicious external scripts.	
Suggested Countermeasures	
• Content-Security-Policy (CSP): Implement a strict CSP to explicitly define which dynamic resources are allowed to load. Example: default-src 'self'; script-src 'self' • X-Frame-Options: Set to DENY or SAMEORIGIN to prevent UI redressing (Clickjacking). • X-Content-Type-Options: Set to nosniff to prevent the browser from attempting to guess the MIME type, which mitigates MIME-sniffing XSS attacks. • Strict-Transport-Security (HSTS): Enforce HTTPS connections globally by setting Strict-Transport-Security: max-age=31536000; includeSubDomains.	
References	
https://owasp.org/Top10/A05_2021-Security_Misconfiguration/https://cwe.mitre.org/data/definitions/1021.html https://cheatsheetseries.owasp.org/cheatsheets/HTTP-Headers-Cheat-Sheet.html	

Proof of concept:

1) Use Curl Tool to get HTTP Headers : Missing HTTP Security Headers

```
(kali㉿kali)-[~]  
$ curl -I http://localhost:3000/#/  
HTTP/1.1 200 OK  
Access-Control-Allow-Origin: *  
X-Content-Type-Options: nosniff  
X-Frame-Options: SAMEORIGIN  
Feature-Policy: payment 'self'  
X-Recruiting: /#/jobs  
Accept-Ranges: bytes  
Cache-Control: public, max-age=0  
Last-Modified: Thu, 01 Oct 2026 12:14:46 GMT  
ETag: W/"24b1-1a0f76379c0"  
Content-Type: text/html; charset=UTF-8  
Content-Length: 9393  
Vary: Accept-Encoding  
Date: Thu, 01 Oct 2026 12:21:09 GMT  
Connection: keep-alive  
Keep-Alive: timeout=5
```

Figure 36 HTTP HEADERS (Missing Security Headers)